

Контрольная работа № 3

Темы:

Настройка и поддержка транзакций в СУБД PostgreSQL

Выполнил студент 2 курса группы
ЭФБО-04-24 Валекжанин В.С.

Проверил доцент Бочаров М.И.

Москва
2026 г.

Оглавление

Часть 1. Вставка, обновление и удаление строки

Цель: Изучение работы с DML-запросами в транзакциях и исследование версионирования строк (xmin, xmax).

Код

```
CREATE TABLE IF NOT EXISTS Users (  
  id SERIAL PRIMARY KEY NOT NULL,  
  name VARCHAR(100) NOT NULL,  
  age INTEGER NOT NULL,  
  gender CHAR NOT NULL  
);
```

```
INSERT INTO Users (name,age,gender) VALUES('Сергей_valekzhanin', 20, 'M');  
SELECT xmin, xmax, * FROM Users ORDER BY id;
```

```
BEGIN;  
  UPDATE Users SET age=age+1;  
END;  
SELECT xmin, xmax, * FROM Users ORDER BY id;
```

```
BEGIN;  
  DELETE FROM Users WHERE id=1;  
END;  
SELECT xmin, xmax, * FROM Users ORDER BY id;
```

```
TRUNCATE Users;  
DROP TABLE Users;
```

Скриншоты выполнения

[Скриншот 1 - Вывод таблицы с xmin]

[Скриншот 2 - Вывод SELECT в обоих терминалах при незафиксированном UPDATE]

[Скриншот 3 - Вывод SELECT при незафиксированном DELETE]

[Скриншот 4 - Вывод после COMMIT]

[Скриншот 5 - Вывод]

[Скриншот 6 - Вывод]

[Скриншот 7 - Вывод]

Часть 2. Видимость версии строки на различных уровнях изоляции

Цель: Исследование уровней изоляции READ COMMITTED и REPEATABLE READ и их влияния на видимость изменений между транзакциями.

Код

```
CREATE TABLE IF NOT EXISTS Users (  
  id SERIAL PRIMARY KEY NOT NULL,  
  name VARCHAR(100) NOT NULL,  
  age INTEGER NOT NULL,  
  gender CHAR NOT NULL  
);  
  
-- Часть 2: Видимость на READ COMMITTED  
BEGIN;  
  SHOW transaction_isolation;  
  INSERT INTO Users (name, age, gender) VALUES('Уровень_Committed', 25, 'M');  
COMMIT;  
SELECT xmin, xmax, * FROM Users ORDER BY id;  
  
-- Часть 2: Видимость на REPEATABLE READ  
BEGIN ISOLATION LEVEL REPEATABLE READ;  
  SHOW transaction_isolation;  
  INSERT INTO Users (name, age, gender) VALUES('Уровень_Repeatable', 30, 'F');  
COMMIT;  
SELECT xmin, xmax, * FROM Users ORDER BY id;  
  
TRUNCATE Users;  
DROP TABLE Users;
```

Скриншоты выполнения

[Скриншот 8 - SHOW transaction_isolation]

[Скриншот 9 - Скриншот аномалии неповторяющегося чтения]

[Скриншот 10 - Скриншот уровня REPEATABLE READ без аномалии]

Часть 3. Состояние транзакции по CLOG

Цель: Изучить статус транзакций (in progress, committed, aborted) и работу механизма фиксации транзакций.

Код

```
CREATE TABLE IF NOT EXISTS Users (  
  id SERIAL PRIMARY KEY NOT NULL,  
  name VARCHAR(100) NOT NULL,  
  age INTEGER NOT NULL,  
  gender CHAR NOT NULL  
);  
  
-- Часть 3: Состояние CLOG с коммитом  
BEGIN;  
  INSERT INTO Users (name, age, gender) VALUES('Для_Коммита', 20, 'F');  
  -- В скрипте txid_current() можно просто вывести, pg_xact_status требует ручной  
  подстановки  
  SELECT txid_current() AS current_tx;  
COMMIT;  
  
-- Часть 3: Состояние CLOG с откатом  
BEGIN;  
  INSERT INTO Users (name, age, gender) VALUES('Для_Отката', 22, 'M');  
  SELECT txid_current() AS current_tx;  
ROLLBACK;  
  
TRUNCATE Users;  
DROP TABLE Users;
```

Скриншоты выполнения

[Скриншот 11 - Статус in progress]

[Скриншот 12 - Статус committed]

[Скриншот 13 - Статус aborted]

Часть 4. Блокировки таблицы

Цель: Рассмотреть блокировки таблиц (RowExclusiveLock и ShareLock) и возникновение ситуаций ожидания.

Код

```
CREATE TABLE IF NOT EXISTS Users (  
  id SERIAL PRIMARY KEY NOT NULL,  
  name VARCHAR(100) NOT NULL,  
  age INTEGER NOT NULL,  
  gender CHAR NOT NULL  
);  
  
INSERT INTO Users (name, age, gender) VALUES('Блокируемый_Пользователь', 45, 'М');  
  
-- Часть 4: Блокировки таблицы  
BEGIN;  
  -- Смотрим свой PID  
  SELECT pg_backend_pid() AS my_pid;  
  
  -- Обновление наложит RowExclusiveLock  
  UPDATE Users SET age = age + 1 WHERE id = 1;  
  
  -- Просмотр блокировок (должна быть RowExclusiveLock)  
  SELECT locktype, transactionid, mode, relation::regclass as obj  
  FROM pg_locks  
  WHERE pid = pg_backend_pid();  
COMMIT;  
  
-- После коммита индекс спокойно создастся (ShareLock)  
CREATE INDEX ON Users (id);  
  
TRUNCATE Users;  
DROP TABLE Users;
```

Скриншоты выполнения

[Скриншот 14 - Вывод блокировок RowExclusiveLock]

[Скриншот 15 - Терминал 2 в ожидании ShareLock]

[Скриншот 16 - Успешное создание индекса]